

Script 1 — Introduction and Context

Slides 1–7 · about 4 minutes

Good morning, respected supervisors and members of the panel. We are Md. Sakib Hossain, Mohammad Nahiyen Kabir and Hasin Mahtab Alvee. Today we will present our thesis, Orthogonal Defect Classification on Defects4J Using an LLM-Driven Scientific Approach. I will start with the context, my teammate will explain our method, and our third teammate will present the results.

Let me start with why this matters. Defects4J is a well-known dataset of real Java bugs. It has 836 bugs from 17 projects, and each bug comes with its bug report, failing tests, the buggy code, the fixed code and the fix. Knowing the type of a bug helps, because different types of bugs need different ways of debugging. Labelling bugs by hand works, but it is slow and does not scale. Language models can read all this evidence quickly. But should a model answer at once, or should it investigate first? And since every bug comes with its fix, we can classify the same bug before and after the fix.

To label bugs, we use Orthogonal Defect Classification, or ODC. We focus on one part of it, the Defect Type, which describes what kind of correction was made. There are seven types in two groups: Checking, Algorithm, Assignment and Timing, and then Function, Interface and Relationship. We also added an "Other" option, which the model can use only if it gives a reason.

Earlier work on Defects4J described how bugs show up in the code, what the patch changed, or looked at about fifty bugs by hand. Language model classifiers mostly read only the bug report, ODC studies used other datasets, and scientific debugging was used to find faults, not to label them. So nobody has given Defects4J real defect types, there is no way to evaluate such labels, and nobody has tested whether investigating helps.

So our problem is this: classify Defects4J bugs into ODC types using only

pre-fix evidence, and evaluate the result without any true labels. To do that, a solution must use all the evidence, focus on meaning, use a fixed set of types, allow "Other", scale up, and bring its own reference. Our answers are one case file per bug, ODC types with "Other", language model strategies, and a post-fix reference.

Our core idea is one bug, two classifications. Take Chart 9, where the fix widens a check to catch an empty range. In pre-fix mode, the model sees only the report, tests, stack trace and code, like a developer before fixing. In post-fix mode, it sees the same evidence plus the fix. Since Defects4J has no ODC labels, the post-fix label becomes our reference answer. We check if the pre-fix label holds, and when the two labels differ, we call that drift. Agreement is not proof of being right; it shows how close the pre-fix label comes to a better-informed one.

This gives us four research questions. First, which defect types are most common, and does that change by project? Second, do the seven types cover every bug, or do we need "Other"? Third, how often does the pre-fix label match the post-fix label? And fourth, what do the ODC taxonomy and our scientific strategy add? Now my teammate will explain how our pipeline works.